

## HW2

邓若辰 PB25111617

### 2.19

已知线性表中的元素以值递增有序排列，并以单链表作存储结构。试写一个高效的算法，删除表中所有值大于mink且小于maxk的元素（若表中存在这样的元素），同时释放被删除结点空间，并分析你的算法的时间复杂度（mink和maxk是给定的两个参变量，它们的值可以和表中的元素相同，也可以不同）。

```
linklist select(linklist &L, ElemType mink, ElemType maxk){
    linklist pre;
    linklist after;
    pre=L;
    after=pre->next;
    while(pre->next!=NULL){
        if((pre->next->data)<=mink){
            pre=pre->next;
            after=after->next;
        }else{
            break;
        }
    }
    while(after!=NULL){
        if(after->data<maxk){
            after=after->next;
        }else{
            break;
        }
    }
    if(pre->next==NULL){
        return L;
    }
    if(pre->next==after){
        return L;
    }
    linklist delete=pre->next;
    pre->next=after;
    while(delete!=after){
        linklist temp=delete->next;
        free(delete);
        delete=temp;
    }
    return L;
}
```

最坏情况下，需要遍历整个单链表，时间复杂度为

$$O(n)$$

## 2.21

试写一算法，实现顺序表的就地逆置，即利用原表的存储空间将线性表  $(a_1, a_2, \dots, a_n)$  逆置为  $(a_n, a_{n-1}, \dots, a_1)$

```
SqList* revrese(SqList &L){
    if(L.length<=1){
        return &L;
    }
    ElemType* front;
    ElemType* last;
    front=L.elem;
    last=front+L.length-1;
    while(front!=last){
        ElemType temp;
        temp=*(front);
        *(front)=*(last);
        *(last)=temp;
        front++;
        last--;
    }
    return &L;
}
```

时间复杂度为

$$O(n)$$

## 2.24

假设有两个按元素值递增有序排列的线性表A和B，均以单链表作为存储结构，请编写算法将A表和B表归并成一个按元素值递减有序（即非递增有序，允许链表中含有值相同的元素）排列的线性表C，并要求利用原表（即A表和B表）的结点空间构造C表。

```
linklist mergelist(linklist &A, linklist &B){
    linklist C=A;
    linklist temp;
    linklist a=A->next;
    linklist b=B->next;
    C->next=NULL;
    while(a!=NULL&&b!=NULL){
        if(a->data<=b->data){
            temp=a;
            a=a->next;
            temp->next=C->next;
            C->next=temp;
        }else{
            temp=b;
            b=b->next;
            temp->next=C->next;
        }
    }
    if(a!=NULL) C->next=a;
    if(b!=NULL) C->next=b;
    return C;
}
```

```

        C->next=temp;
    }
}
while(a!=NULL){
    temp=a;
    a=a->next;
    temp->next=C->next;
    C->next=temp;
}
while(b!=NULL){
    temp=b;
    b=b->next;
    temp->next=C->next;
    C->next=temp;
}
return C;
}

```

时间复杂度：

$$O(n + m)$$

## 2.29

已知A,B,C为三个递增有序的线性表，现要求对A表做如下操作：删去那些既在B表中出现又在C表中出现的元素。试对顺序表编写实现上述操作的算法，并分析你算法的时间复杂度（注意：题目中没有特别指明同一表中的元素值互不相同）

```

SqList* redo(SqList &A, SqList &B, SqList &C){
    int i,j,k;
    int cnt=0;
    i=0;
    j=0;
    k=0;
    while(i<A.length){
        while(j<B.length&&*(B.elem+j)<*(A.elem+i)){
            j++;
        }
        while(k<C.length&&*(C.elem+k)<*(A.elem+i)){
            k++;
        }
        if(j<B.length&&k<C.length&&*(B.elem+j)==*(A.elem+i)&&*(C.elem+k)==*(A.elem+i)){
            int val=*(A.elem+i);
            while(i<A.length&&*(A.elem+i)==val){
                i++;
            }
        }else{
            *(A.elem+cnt)=*(A.elem+i);
            i++;
        }
    }
}

```

```
        cnt++;  
    }  
}  
A.length=cnt;  
return &A;  
}
```

最坏情况下要遍历三个顺序表，时间复杂度：

$$O(n + p + q)$$